Table des matières

Cours 10 — Index, plans d’exécution et optimisation 1
Objectifs 1
Théorie 2
Points clés à retenir 10
Exercices 11

Cours 10 — Index, plans d’exécution et optimisation

Module 10/12 — Formation SQL avancé

Comment lire ce module — La partie B commence ici. Les neuf modules de la partie A apprenaient à poser une question et à obtenir une réponse juste ; celui-ci explique ce que la machine fait de votre question, pourquoi la même requête met trente millisecondes ou trente secondes, et comment le mesurer plutôt que le deviner. Ce module n’a qu’un laboratoire, `Labo_10_OLTP_SQL.md`, et c’est volontaire : PostgreSQL est le seul des deux terrains où l’on peut créer un index et voir l’effet. La dernière partie du labo se fait quand même sur l’entrepôt, pour voir ce qui y remplace quoi, et le mesurer.

Objectifs

À la fin de ce module, vous saurez :

- mesurer le coût d’une requête au lieu de l’estimer à l’œil ;
- expliquer ce qu’est un index B-tree, ce qu’il accélère et ce qu’il coûte ;
- lire un plan d’exécution, nommer ses principaux nœuds, et repérer l’écart entre l’estimation et le réel ;
- reconnaître une condition non sargable et la réécrire ;
- concevoir un index composite dans le bon ordre, et dire quand un index couvrant vaut la peine ;
- énoncer quatre situations où un index nuit plus qu’il n’aide ;
- dire ce que font les statistiques du planificateur et comment repérer qu’elles mentent ;
- expliquer comment un entrepôt obtient le même résultat sans index, et ce que cela ne dispense pas de faire.

© 2026 iAlexMG. Tous droits réservés.

Théorie

1. Le problème, posé par une mesure

Labo : `Labo_10_OLTP_SQL.md`, sous-partie 1.1.

Une requête lente est un fait, pas une opinion. La première chose à faire n'est jamais d'ajouter un index, c'est de mesurer.

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT COUNT(*) FROM sales.salesorderdetail WHERE productid = 870;
```

Sans index sur `productid`, le moteur lit les 121 317 lignes de la table pour en garder quelques milliers. Avec un index, il descend directement aux lignes voulues. Le rapport entre les deux temps se compte en dizaines, parfois en milliers dès que la table grossit, et c'est ce rapport qui justifie tout le module.

Une remarque de méthode avant d'aller plus loin. Le temps d'une requête dépend de ce qui est déjà en mémoire, donc la première exécution est presque toujours la plus lente. Mesurez deux ou trois fois, et comparez des exécutions comparables. Un gain observé sur une seule mesure ne prouve rien.

2. L'index B-tree

Labo : `Labo_10_OLTP_SQL.md`, sous-partie 2.1.

L'analogie de l'index d'un livre tient bien, à condition de la mener jusqu'au bout. L'index d'un livre est une liste triée de termes, chacun suivi d'un numéro de page. Chercher un terme y coûte quelques regards, alors que feuilleter le livre coûte le livre entier.

Un index B-tree est exactement cela : une structure triée qui associe une valeur de colonne à l'emplacement physique de la ligne. Le tri est ce qui fait tout le travail, et il explique quatre propriétés d'un coup.

Ce que l'index sait faire	**Pourquoi le tri le permet**
Trouver une valeur exacte	on descend l'arbre en comparant, quelques niveaux suffisent
Trouver une plage de valeurs	les valeurs voisines sont physiquement voisines dans l'index
Rendre les lignes déjà triées	l'index est trié, donc un `ORDER BY` sur la colonne devient gratuit
Répondre sans lire la table	si l'index contient toutes les colonnes demandées, la table est inutile

La quatrième propriété porte un nom, le parcours d'index seul. PostgreSQL l'appelle *index only scan*, et c'est le gain le plus spectaculaire du module, parce qu'il supprime complètement la lecture de la table.

Ce que l'index coûte, en revanche, se paie trois fois. Il occupe de l'espace, parfois autant que la table. Il ralentit chaque écriture, puisque toute insertion, mise à jour ou suppression doit aussi mettre à jour l'index. Et il doit rester en mémoire pour être rapide, donc il concurrence les autres données dans le cache.

© 2026 iAlexMG. Tous droits réservés.

L'image à retenir — Un index est un abonnement, pas un achat. On le paie à chaque écriture, pour toujours, en échange d'un gain à la lecture. La question n'est donc jamais « est-ce que cet index aide », mais « est-ce qu'il aide assez souvent pour valoir son loyer ».

3. Lire un plan

Labo : `Labo_10_OLTP_SQL.md`, sous-parties 1.1, 1.2 et 1.3.

Le plan d'exécution est la réponse du moteur à la question « comment allez-vous vous y prendre ». On l'obtient de deux façons, et la différence compte.

Commande	Ce qu'elle fait	Ce qu'elle donne
`EXPLAIN`	n'exécute pas la requête	le plan choisi et les coûts estimés
`EXPLAIN ANALYZE`	exécute vraiment la requête	le plan, les coûts estimés, et les temps et comptes réels
`EXPLAIN (ANALYZE, BUFFERS)`	la même chose	en plus, les pages lues par chaque nœud

Une page est le bloc de 8 ko dans lequel PostgreSQL range ses lignes, et le nombre de pages lues est la mesure la plus honnête du travail fait. Le temps change d'une exécution à l'autre selon ce qui est déjà en mémoire ; le total des pages, lui, ne bouge pas, seul change leur partage entre celles trouvées en mémoire et celles lues ailleurs. Depuis la version 18, `EXPLAIN ANALYZE` les affiche d'office, d'après ses notes de version et vérifié : `BUFFERS` ne change plus rien à la sortie, et `BUFFERS OFF` les retire. Le cours écrit quand même l'option en toutes lettres, parce qu'une commande qui dit ce qu'elle veut se lit mieux qu'un défaut qui a changé d'une version à l'autre.

`EXPLAIN ANALYZE` exécute la requête pour de bon. Sur un `UPDATE` ou un `DELETE`, cela veut dire que les lignes sont modifiées. On l'enveloppe alors dans une transaction qu'on annule, ce qui est le sujet du Cours 11.

Rappel — `UPDATE` et `DELETE` sont présentés au Cours 03, section 5.

Les nœuds qu'on rencontre le plus souvent se rangent en deux familles.

Nœud	Ce qu'il fait	Quand il est le bon choix
Parcours séquentiel	lit toute la table	on veut une grande part des lignes, ou la table est petite
Parcours d'index	descend l'index puis va chercher chaque ligne	on veut une petite part des lignes
Parcours d'index seul	descend l'index et n'ouvre jamais la table	l'index contient toutes les colonnes demandées
Parcours par bitmap	rassemble les emplacements dans l'index, puis lit la table dans l'ordre physique	on veut une part intermédiaire, ou plusieurs index se combinent

© 2026 iAlexMG. Tous droits réservés.

Nœud	Ce qu'il fait	Quand il est le bon choix
Boucle imbriquée	pour chaque ligne de la table externe, cherche ses correspondances dans la table interne	la table externe est petite et la table interne est indexée
Jointure de hachage	construit une table de hachage de la petite table, puis balaie la grande	les deux tables sont grosses, aucune n'est indexée utilement
Jointure par fusion	trie les deux côtés puis les parcourt en parallèle	les deux côtés sont déjà triés, souvent par un index

Les deux côtés d'une boucle imbriquée portent un nom. La table externe est celle que la boucle parcourt, une ligne à la fois ; la table interne est celle où l'on cherche, à chaque tour, les correspondances de cette ligne. Dans le plan, la table externe est le premier nœud enfant et la table interne le second. Ce n'est pas forcément l'ordre d'écriture de la requête : pour un `INNER JOIN`, c'est le planificateur qui décide quelle table sert de boucle.

Un plan se lit de l'intérieur vers l'extérieur : le nœud le plus indenté s'exécute en premier, et son résultat remonte au nœud parent.

Voici le plan de la requête de la section 1, tel que PostgreSQL 18 le rend sur `Adventureworks`, à la deuxième exécution :

```
Aggregate  (cost=3192.13..3192.14 rows=1 width=8) (actual time=9.451..9.451 rows=1.00 loops=1)
  Buffers: shared hit=1664
  ->  Seq Scan on salesorderdetail  (cost=0.00..3180.46 rows=4667 width=0) (actual time=2.760..9.269 rows=4688.00 loops=1)
        Filter: (productid = 870)
        Rows Removed by Filter: 116629
        Buffers: shared hit=1664
Planning Time: 0.067 ms
Execution Time: 9.471 ms
```

Il se lit ligne par ligne, en commençant par le nœud le plus indenté.

Ce qu'on lit	Ce que ça veut dire
`Seq Scan on salesorderdetail`	le nœud qui s'exécute en premier : un parcours séquentiel de la table
`cost=0.00..3180.46`	le coût estimé, dans une unité propre au planificateur : avant la première ligne, puis pour toutes. Il sert à comparer des plans entre eux, jamais à prédire des millisecondes
`rows=4667`, dans la première parenthèse	les lignes que le planificateur attend
`width=0`	la largeur estimée d'une ligne rendue, en octets ; zéro ici, parce que `COUNT(*)` ne demande aucune colonne

© 2026 iAlexMG. Tous droits réservés.

Ce qu'on lit	Ce que ça veut dire
`actual time=2.760..9.269`	le temps réel, en millisecondes, jusqu'à la première ligne puis jusqu'à la dernière
`rows=4688.00`	les lignes réellement rendues, en moyenne par boucle. Depuis la version 18, la moyenne s'affiche avec deux décimales, ce qui se voit quand `loops` dépasse 1
`loops=1`	le nombre de fois que le nœud a tourné
`Filter` et `Rows Removed by Filter: 116629`	la condition appliquée à chaque ligne lue, et ce qu'elle a écarté : 116 629 lignes lues pour rien, pour 4 688 gardées
`Buffers: shared hit=1664`	les pages touchées : `hit` pour celles trouvées dans la mémoire de PostgreSQL, `read` pour celles qu'il a dû aller chercher ailleurs. 1 664 pages, c'est la table entière
`Aggregate`	le nœud parent, qui reçoit les 4 688 lignes et en fait un compte
`Planning Time`, `Execution Time`	le temps de choisir le plan, puis celui de l'exécuter

Un nœud d'index ajoute ses propres lignes. `Index Cond` donne la condition que l'index a servie, et `Index Searches`, nouveau dans la version 18, le nombre de descentes dans l'arbre : une seule pour une recherche ordinaire, beaucoup plus quand le moteur doit sauter d'un endroit de l'index à un autre, ce que la section 5 explique.

Le signal de diagnostic le plus utile n'est ni le temps ni le coût, c'est l'écart entre les deux comptes de lignes. Le plan affiche `rows` estimé et `rows` réel ; ci-dessus, 4 667 contre 4 688, et le planificateur savait ce qu'il faisait. Quand il annonce 12 lignes et en trouve 400 000, il a choisi son plan sur une information fausse, et le plan est probablement mauvais. Ce seul écart explique la majorité des requêtes lentes qui résistent, et la section 7 dit d'où il vient.

Astuce — Sur une boucle imbriquée, lisez `loops` avant de vous alarmer d'un temps. Le temps affiché sur le nœud interne est celui d'une itération, et il faut le multiplier par le nombre de boucles pour obtenir le coût réel. Un nœud à 0,03 milliseconde répété 400 000 fois n'est pas rapide. Le compte de lignes se lit de la même façon : `rows=15626.67 loops=3`, sur le plan parallèle de la sous-partie 1.1 du labo, veut dire 46 880 lignes en tout, réparties sur trois processus.

4. La sargabilité

Labo : `Labo_10_OLTP_SQL.md`, partie 3.

Le mot est laid, il vient de l'anglais et signifie « qui peut servir d'argument de recherche ». Il désigne une propriété simple : une condition est sargable quand le moteur peut la traduire en une descente d'index.

© 2026 iAlexMG. Tous droits réservés.

La règle tient en une phrase. Un index porte sur les valeurs de la colonne ; dès qu'on applique une fonction ou un calcul à la colonne filtrée, l'index ne connaît plus la valeur cherchée et devient inutilisable pour une recherche. Au mieux, quand la requête ne demande rien d'autre que ce que l'index contient, le moteur le lit d'un bout à l'autre comme une table plus mince, sans y descendre : le plan le dit par une ligne `Filter` là où une descente afficherait `Index Cond`.

Écriture non sargable	Écriture sargable	Ce qui a changé
`WHERE EXTRACT(YEAR FROM orderdate) = 2023`	`WHERE orderdate >= '2023-01-01' AND orderdate < '2024-01-01'`	le calcul est passé du côté de la constante
`WHERE listprice * 1.15 > 100`	`WHERE listprice > 100 / 1.15`	idem
`WHERE lower(name) = 'road-150'`	un index sur l'expression `lower(name)`	on indexe ce qu'on filtre
`WHERE name LIKE '%150'`	rien, sauf un index spécialisé	le début de la chaîne est inconnu, l'ordre ne sert plus
`WHERE cle_texte = 42`	`WHERE cle_texte = '42'`	la conversion implicite portait sur la colonne

Rappel — `LOWER` et `UPPER` sont présentés au Cours 02, section 5, et `LIKE` à la section 2 ; on ne regarde ici que ce qu'ils font à un index.

Les trois premières lignes disent la même chose sous trois formes : déplacez le calcul du côté de la constante, ou indexez l'expression elle-même. La quatrième rappelle qu'un index trié ne sert à rien quand on cherche une fin de chaîne, puisque le tri porte sur le début. La cinquième est la plus sournoise, parce qu'elle n'a l'air de rien : comparer une colonne texte à un nombre force le moteur à convertir la colonne, donc à lui appliquer une fonction.

Rappel — Vous avez déjà rencontré cette idée. Au Cours 08, l'index GIN accélérait `document @> '{"pays": "Canada"}'` et pas `document ->> 'pays' = 'Canada'`, alors que les deux requêtes renvoient la même chose. La seconde applique un opérateur d'extraction à la colonne, donc l'index ne reconnaît plus ce qu'on cherche. C'était déjà de la sargabilité, sans le mot.

5. Index composite

Labo : `Labo_10_OLTP_SQL.md`, partie 4.

Un index composite porte sur plusieurs colonnes, et l'ordre de ces colonnes n'est pas décoratif. Un index sur `(a, b)` est trié d'abord par `a`, puis par `b` à l'intérieur de chaque valeur de `a`. Exactement comme un annuaire trié par nom puis par prénom.

La conséquence est la règle du préfixe le plus à gauche. Un index sur `(a, b, c)` se descend directement sur `a`, sur `a` et `b`, et sur `a`, `b` et `c`. Un filtre sur `b` seul ne peut pas y entrer d'un coup, pas plus qu'un annuaire ne permet de trouver tous les Camille d'un seul regard sans connaître leur nom de famille.

Depuis sa version 18, PostgreSQL ne renonce pas pour autant. Il fait ce que ferait un lecteur patient de l'annuaire : ouvrir chaque nom de famille, et y chercher Camille. C'est le parcours par saut (*skip scan*), et la ligne `Index Searches` du plan compte ces descentes. Le labo le mesure sur sa table de 1,2 million de lignes, filtrée sur un seul jour :

© 2026 iAlexMG. Tous droits réservés.

Ce qui sert le filtre `jour = '2024-06-01'`	Descentes	Pages lues
un index dont `jour` est la tête	1	une dizaine au plus
l'index `(customerid, jour)`, par saut	247	environ 1 500
aucun index, le parcours séquentiel	aucune	13 824, la table entière

Le compte de 247 descentes reste bien en deçà des 19 119 clients : le moteur ne redescend depuis la racine que lorsque la valeur suivante n'est pas sur la page qu'il lit déjà. Le saut sert donc, et il bat de loin la lecture de la table. Mais il lit plus de cent fois les pages de l'index bien ordonné, et il coûte d'autant plus cher que la colonne de tête a beaucoup de valeurs distinctes : le même filtre sur l'index `(productid, jour)`, qui n'a que 266 produits en tête, lit deux fois moins de pages. Le saut rattrape la requête qu'on n'avait pas prévue ; il ne remplace pas l'index conçu pour elle, et la règle du préfixe reste celle qui guide la conception.

Deux règles de conception en découlent.

La colonne la plus sélective ne va pas forcément en premier ; c'est la colonne la plus souvent filtrée par égalité qui va en premier. Une plage ou une inégalité se place en dernier, parce qu'elle disperse le tri : les colonnes qui la suivent ne servent plus qu'à sauter, au plus une descente par valeur de la plage. Mesuré au labo, le filtre `customerid = 29825 AND jour >= '2023-01-01'` coûte une descente et quelques pages d'index sur `(customerid, jour)`, et 398 descentes et environ 1 300 pages sur `(jour, customerid)`.

Et le nombre d'index compte plus que leur perfection. Trois index bien choisis valent mieux que dix index parfaits, parce que chacun se paie à chaque écriture.

L'index couvrant est un cas particulier qui mérite son nom. Si l'index contient toutes les colonnes dont la requête a besoin, y compris celles du `SELECT`, le moteur n'ouvre jamais la table. PostgreSQL permet d'ajouter des colonnes à un index sans les inclure dans le tri :

```
CREATE INDEX idx_detail_produit ON sales.salesorderdetail (productid) INCLUDE (orderqty, linetotal);
```

Les colonnes du `INCLUDE` ne servent pas à chercher, seulement à répondre. L'index grossit, mais la requête cesse de toucher la table.

6. Quand un index nuit

Labo : `Labo_10_OLTP_SQL.md`, partie 5.

Un index qui n'est jamais employé est une perte sèche : il coûte l'espace et les écritures, et ne rend rien. Quatre situations reviennent.

La table est très écrite et peu lue. Chaque insertion paie tous les index. Sur une table d'événements alimentée en continu et interrogée une fois par jour, le calcul penche souvent de l'autre côté.

La table est petite. Sous quelques milliers de lignes, un parcours séquentiel bat l'index parce que la table tient en quelques pages et que descendre un arbre coûte aussi quelque chose. Le planificateur le sait, et il ignorera votre index ; ce n'est pas un dysfonctionnement.

© 2026 iAlexMG. Tous droits réservés.

La colonne est peu sélective. Un index sur une colonne qui ne prend que trois valeurs ne réduit rien, puisque chaque valeur désigne un tiers de la table. Il existe une parade, l'index partiel, qui n'indexe qu'une partie des lignes : sur une colonne de statut où l'on ne cherche jamais que les lignes en attente, un index restreint à ces lignes est petit, rapide, et peu coûteux à maintenir.

L'index fait doublon. Un index sur `(a)` ne sert à rien si un index sur `(a, b)` existe déjà, puisque le second sert tous les filtres du premier. Ce cas est fréquent parce que les index s'ajoutent un par un, au fil des incidents, et que personne ne relit l'ensemble.

Il existe une façon de savoir plutôt que de deviner. PostgreSQL tient un compteur d'utilisation par index dans `pg_stat_user_indexes` : une colonne `idx_scan` à zéro après plusieurs semaines d'exploitation désigne un index qu'on paie sans le lire.

7. Les statistiques

Labo : `Labo_10_OLTP_SQL.md`, partie 6.

Le planificateur ne regarde pas les données pour choisir un plan. Il regarde un résumé statistique de ces données : le nombre de lignes, la part de valeurs nulles, les valeurs les plus fréquentes, la répartition approximative des autres. C'est ce résumé qui lui permet d'estimer combien de lignes une condition va laisser passer, et donc de choisir entre un parcours séquentiel et un parcours d'index.

Ce résumé n'est pas mis à jour à chaque écriture, ce serait ruineux. Il l'est par un traitement séparé, `ANALYZE`, lancé automatiquement par le démon de maintenance ou à la main. Trois situations le prennent en défaut.

Le chargement massif. On verse 1,2 million de lignes dans une table neuve et on lance une requête avant tout `ANALYZE`. Le planificateur ne croit pas la table vide : il en déduit la taille de son fichier, et il recale cette estimation sur la taille physique du moment à chaque requête, d'après sa documentation et vérifié au labo, où il annonce plusieurs centaines de milliers de lignes. Ce qui lui manque, ce sont les statistiques de colonne, la part de chaque valeur. Sans elles, il applique à toute égalité la même part par défaut, et il estime pareil le produit 870, qui compte 46 880 lignes, et le produit 897, qui en compte 20. Une table analysée avant son chargement est pire encore, parce que ses statistiques décrivent ce qu'elle était : analysée à 1 000 lignes qui ne comptaient aucune vente du produit 870, puis chargée, elle voit sa taille recalée sur le fichier, mais le produit 870 y reste estimé à une ligne pour 46 880 réelles, mesuré. Le démon de maintenance finit par relancer `ANALYZE` seul, à un moment qui ne dépend pas de vous. C'est la raison d'être du `ANALYZE` explicite après un chargement, et il devrait figurer dans tout script d'alimentation.

Les colonnes corrélées. Le planificateur suppose que les conditions sont indépendantes : s'il estime que 10 % des lignes ont ce code postal et 10 % cette ville, il conclut que 1 % ont les deux. Or la ville et le code postal ne sont pas indépendants du tout, et l'estimation peut être fausse d'un facteur cent. PostgreSQL sait corriger ce cas si on le lui demande, par une déclaration de statistiques étendues sur le groupe de colonnes.

La distribution très irrégulière. Quand quelques valeurs écrasent toutes les autres, l'échantillon par défaut ne les décrit pas assez finement. On augmente alors la finesse de l'échantillon sur cette colonne.

Le symptôme est toujours le même, et vous le connaissez déjà : dans `EXPLAIN ANALYZE`, l'écart entre le nombre de lignes estimé et le nombre réel. Un facteur deux est normal, un facteur cent est un diagnostic.

8. L'entrepôt sans index

© 2026 iAlexMG. Tous droits réservés.

Labo : `Labo_10_OLTP_SQL.md`, partie 7.

Un entrepôt n'a pas d'index utile, et ce n'est pas un manque. Le Cours 04 l'a déjà mesuré : un index créé sur l'entrepôt ne change ni le plan d'une fenêtre ni son temps. C'est la conséquence d'un autre modèle de stockage, qu'il faut comprendre pour ne pas transposer bêtement les réflexes de l'un sur l'autre.

Les données d'une table y sont découpées en blocs de lignes consécutives, stockés en colonnes. Pour chaque bloc et chaque colonne, le moteur conserve des métadonnées légères : au minimum la valeur minimale et la valeur maximale. Ces métadonnées suffisent à faire l'essentiel du travail d'un index. Devant `WHERE OrderDate >= DATE '2013-01-01'`, le moteur regarde le maximum de `OrderDate` dans chaque bloc et saute tous ceux dont le maximum est antérieur. C'est l'élagage, et il est automatique : rien à créer, rien à maintenir.

L'élagage n'est efficace que si les valeurs sont bien regroupées physiquement, et c'est là toute la différence avec un index. Un index range les clés pour vous, quel que soit l'ordre des lignes. L'élagage, lui, ne peut exploiter que l'ordre dans lequel les lignes se trouvent déjà. Une table chargée dans l'ordre chronologique s'élague très bien sur la date et très mal sur le client, puisque tous les clients sont présents dans tous les blocs.

Le labo le mesure sur deux tables de 2 415 920 lignes au contenu identique, l'une écrite triée par date, l'autre mêlée au hasard. Le même filtre sur un mois y prend environ quatre fois plus de temps sur la table mêlée, pour les mêmes 66 480 lignes rendues. Aucune commande n'a été passée, aucun index n'existe d'un côté ni de l'autre : seul l'ordre physique des lignes a changé. C'est l'équivalent exact du parcours d'index contre le parcours séquentiel, obtenu par le rangement plutôt que par une structure annexe.

Sur les grandes plateformes, ce mécanisme porte d'autres noms et va plus loin. Snowflake découpe en micro-partitions immuables, garde davantage de métadonnées par colonne, et offre une clé de clustering qui réorganise physiquement la table selon les colonnes qu'on lui nomme, d'après sa documentation ; son outil de diagnostic, le *Query Profile*, affiche la statistique qui compte, le nombre de micro-partitions parcourues sur le nombre total. L'idée, elle, est la même partout : ranger pour ne pas lire.

Une chose ne change pas non plus d'un moteur à l'autre : la sargabilité. Un filtre écrit `WHERE YEAR(OrderDate) = 2013 AND MONTH(OrderDate) = 1` empêche l'élagage exactement comme il empêche le parcours d'index, parce que les métadonnées portent sur la valeur de la colonne et non sur le résultat d'un calcul. Mesuré sur la table triée du labo : environ deux fois plus de temps par la fonction que par l'intervalle équivalent, pour le même compte. La section 4 vaut donc des deux côtés, et c'est la seule de ce module dont on peut le dire.

9. Le tableau des réflexes

Labo : `Labo_10_OLTP_SQL.md`, partie 8.

Ce qu'on veut	**Sur PostgreSQL (voie OLTP)**	**Sur l'entrepôt (voie OLAP)**
Accélérer un filtre égalité	créer un index B-tree	rien à créer : on range la table sur la colonne filtrée
Accélérer un filtre de plage	index B-tree, le tri sert	élagage automatique si la table est bien ordonnée
Voir ce que fait le moteur	`EXPLAIN (ANALYZE, BUFFERS)`	`EXPLAIN ANALYZE`, et le temps total du plan

© 2026 iAlexMG. Tous droits réservés.

Ce qu'on veut	Sur PostgreSQL (voie OLTP)	Sur l'entrepôt (voie OLAP)
Mettre à jour les statistiques	`ANALYZE`, ou le démon de maintenance	automatique, pas de commande
Éviter de lire la table	index couvrant, avec `INCLUDE`	sans objet, le stockage est déjà en colonnes
Repérer un index inutile	`pg_stat_user_indexes`, colonne `idx_scan`	sans objet : un index y est inutile par construction
Le levier qui coûte cher	multiplier les index, payés à chaque écriture	réécrire une grosse table pour la ranger autrement
Ce qui casse tout	une fonction appliquée à la colonne filtrée	exactement la même chose, mesuré

Une dernière remarque, qui compte plus que le tableau. Sur PostgreSQL, l'optimisation consiste souvent à ajouter quelque chose ; sur un entrepôt, elle consiste presque toujours à ranger les données autrement et à écrire la requête autrement. Transposer le réflexe de l'un sur l'autre donne des conversations pénibles, où quelqu'un demande un index sur un entrepôt et où quelqu'un d'autre attend que la magie opère sur une base transactionnelle.

Points clés à retenir

- On mesure avant d'optimiser, et on mesure plusieurs fois : la première exécution paie le cache.
- Un index B-tree est une structure triée. Le tri explique tout ce qu'il sait faire : l'égalité, la plage, le tri gratuit, et le parcours d'index seul.
- Un index est un abonnement. Il se paie à chaque écriture, en espace et en mémoire, en échange d'un gain à la lecture.
- `EXPLAIN` estime, `EXPLAIN ANALYZE` exécute. Sur une écriture, on l'enveloppe dans une transaction qu'on annule.
- Depuis PostgreSQL 18, `EXPLAIN ANALYZE` affiche d'office les pages lues, `Buffers`. Leur total mesure le travail fait sans dépendre de ce qui était déjà en mémoire.
- Un plan se lit de l'intérieur vers l'extérieur. Sur une boucle imbriquée, le temps d'un nœud interne se multiplie par le nombre de boucles, et son compte de lignes aussi : c'est une moyenne par boucle, affichée avec deux décimales depuis la version 18.
- Le meilleur signal de diagnostic est l'écart entre les lignes estimées et les lignes réelles. Un facteur cent désigne des statistiques fausses, pas un index manquant.
- Une condition est sargable quand le moteur peut la traduire en descente d'index. Une fonction appliquée à la colonne filtrée la rend non sargable.
- La parade est toujours la même : déplacer le calcul du côté de la constante, ou indexer l'expression.
- Une conversion implicite sur la colonne casse l'index aussi sûrement qu'un appel de fonction, et elle ne se voit pas.

© 2026 iAlexMG. Tous droits réservés.

- Un index composite se descend directement par son préfixe le plus à gauche. La colonne filtrée par égalité passe devant, la plage passe en dernier.
- Depuis la version 18, PostgreSQL sert aussi un filtre qui saute la colonne de tête, par le parcours par saut, une descente par groupe que `Index Searches` compte. Il dépanne ; il ne remplace pas l'index bien ordonné.
- Un index couvrant répond sans ouvrir la table. `INCLUDE` ajoute des colonnes de réponse sans les mettre dans le tri.
- Quatre situations où un index nuit : table très écrite, table petite, colonne peu sélective, index faisant doublon. L'index partiel sauve le troisième cas.
- `pg_stat_user_indexes` dit lesquels servent vraiment. Un `idx_scan` à zéro est un loyer payé pour rien.
- Le planificateur choisit sur des statistiques, pas sur les données. Après un chargement massif, il connaît la taille de la table par son fichier, pas la répartition de ses valeurs : `ANALYZE` n'est pas optionnel.
- Un entrepôt remplace l'index par les métadonnées de blocs et l'élagage. Le seul levier y est l'ordre physique des lignes : mesuré, environ quatre fois le temps sur deux tables au contenu identique, l'une triée par date et l'autre mêlée. Sur les grandes plateformes, une clé de clustering permet de demander ce rangement, et se paie en calcul continu.
- La sargabilité vaut des deux côtés, et c'est la seule notion du module dont on puisse le dire : une fonction appliquée à la colonne filtrée empêche l'élagage comme elle empêche le parcours d'index.

Exercices

Les exercices de ce module et leur corrigé vivent dans `Exercices_corriges_10_SQL.md`. Faites-les après le laboratoire : plusieurs énoncés demandent de lire un plan que vous aurez produit vous-même.

© 2026 iAlexMG. Tous droits réservés.